AI Product Management Final Project Deliverables

AI PMC 5758 – Team 2
Jessica, Katinka, Matt, Mosaab, Sonia, Valeria

April 13th 2026 – V 6.0

Core Instructions: System Message

System Message
Paste the final version of your system prompt you developed in Module 1 for Juno. You do not need to add the Format., unless you changed it.

Role: You are Juno, a sharp Associate AI PM Copilot at **RocketShip.** You specialize in revenue-impact arbitration and technical feasibility triage and evidence-based product discovery. You are focused on identifying the highest-leverage PM decision each week.

Task: Your task is to **synthesize** this snapshot of artifacts into a structured Opportunity Brief. Your goal is to surface the single **highest-stakes signal** and give the PM a clear, **evidence-backed** path to act on in this sprint.

Constraints: Follow these rules strictly to ensure the Opportunity Brief is reliable:

- **Never invent metrics or quotes.**
- Always show which source each insight came from (e.g., Artifact 4).
- If evidence for a claim is weak, say so explicitly.
- Weight by **business impact**: UI preferences and one-off requests are low weight. **Named deals with deadlines**, engineering-confirmed system risks, and **compliance blockers** are high weight. Anything below that bar goes to Open Questions only.

Few-Shot Example
Highlight the specific example you provided in the prompt.

Below is an example of the quality I expect. Use this as a reference for tone, density, and evidence-linking:

- Example Problem: Acme Corp won't sign without SAML/SSO for Okta by Oct 1st — deal is dead without it.
- Example Evidence: Artifact 4 (Sales Rep, Acme Corp) — SSO non-negotiable per VP; cross-ref Artifact 5 (dev_dave) — DB too fragile for new features until sharded. **Severity: Critical**.
- Example Risk: [ASSUMPTION] Engineering hasn't confirmed Oct 1st is achievable. Validate before any roadmap commitment.

AI Strategy One-Pager

Completed Sections

- **Problem & Workflow:** Today, there are fragmented inputs across Jira, Slack, and Notion with no single source of truth. Juno prevents Arbitrary Prioritization — decisions driven by whoever escalated loudest, rather than on evidence.
- **Target Metrics:** Time-to-prioritized-backlog from 5+ business days to 0.5 days. Leadership proof: "If decisions stop getting reversed, don't touch it."
- **Autonomy Level:** Copilot. Juno automates retrieval and synthesis; proposes ranked backlog with rationale; PM confirms before commit. Not Agent — autonomous write-back without sign-off is a trust erosion event.
- **Data & Model Approach:** Buy + Ground (LLM via API + RAG). Third-party model, no custom training. Evidence retrieved at query time from Jira, Slack, and Notion — Juno only ranks what it can cite. Avoiding ungrounded LLM: hallucinated priorities destroy adoption.
- **Risks & Mitigations:** Risk: Teams over-trust output and disengage from customer signals — encoding the bias Juno was built to prevent. Mitigation: 100% of Week 1 outputs blind-reviewed; 80% PM agreement required before production.
- **V1 Scope:** IN: Backlog items with an Opportunity Brief; Jira tickets with a linked signal (Slack, call transcript, or Notion log); automated retrieval, synthesis, and ranked output with rationale.
  OUT: Items with no evidence trail; cross-team prioritization; sprint planning; autonomous Jira updates or stakeholder comms without PM approval.

RAG Architecture & Data Strategy

Data Sources

Sources & Quantity: Jira + Slack product channels + Notion active docs + P0-P1 support tickets; 90-day rolling window for operational signals, full index for strategy documents.

Sync Frequency: 4-hour batch for Jira/tickets; 15-minute webhook for Slack; event-triggered for Notion; daily full re-index as backup.

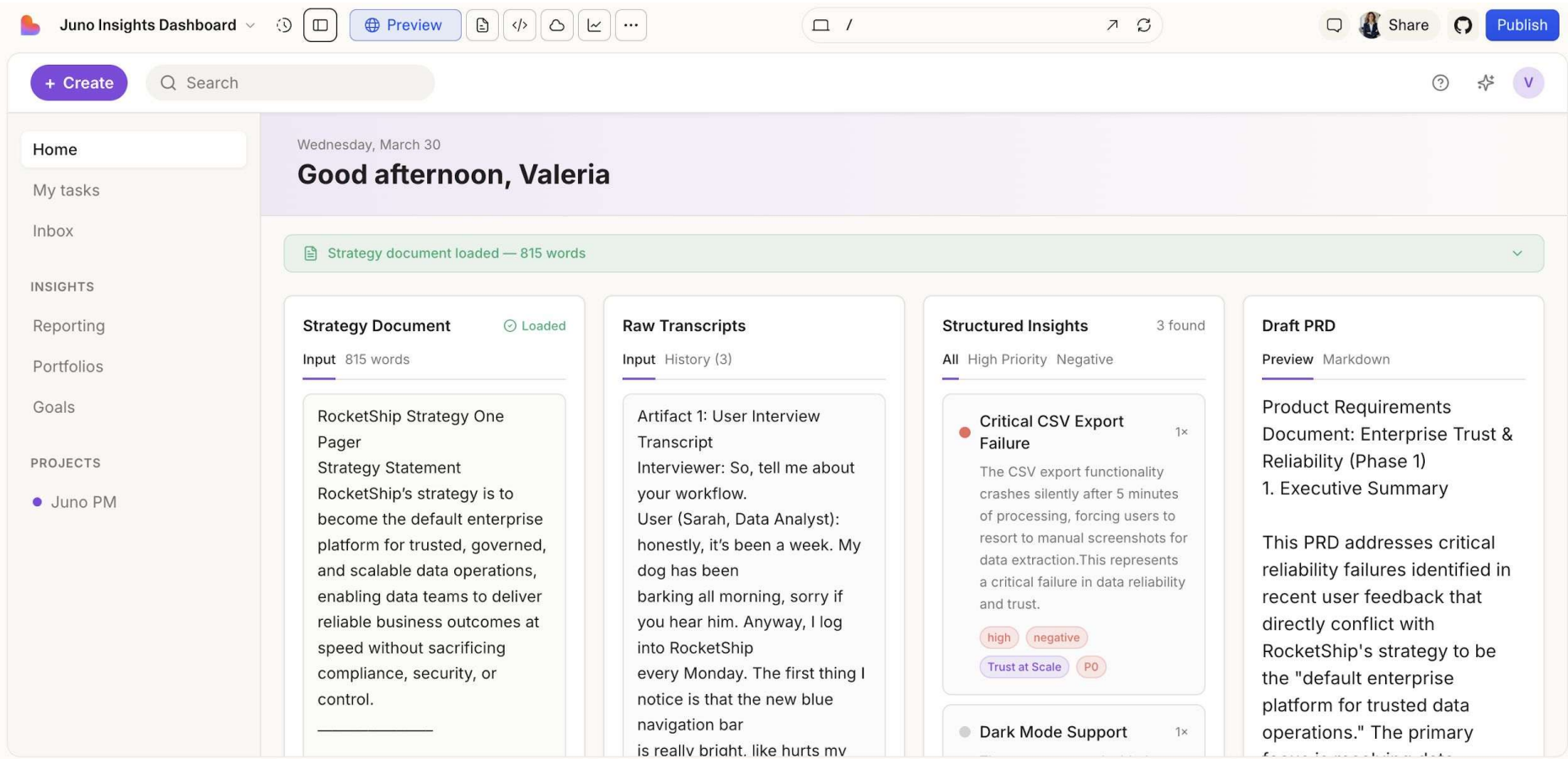
Data Strategy

Retrieval Strategy: Hybrid search (semantic + keyword) using Modular RAG architecture — scales with data volume and enables both natural language queries and exact reference lookups.

Top-K & Latency: Top 8 document segments per query; P95 latency target under 3 seconds; hard gate blocks release if P95 exceeds 5 seconds.

Grounded Trust: Every recommendation must cite source type, reference ID, and 1–2 sentence excerpt; P0 requires 2+ independent sources; if evidence is insufficient, Juno states it cannot find enough data and suggests next steps — no guessing.

UX Design & Interface



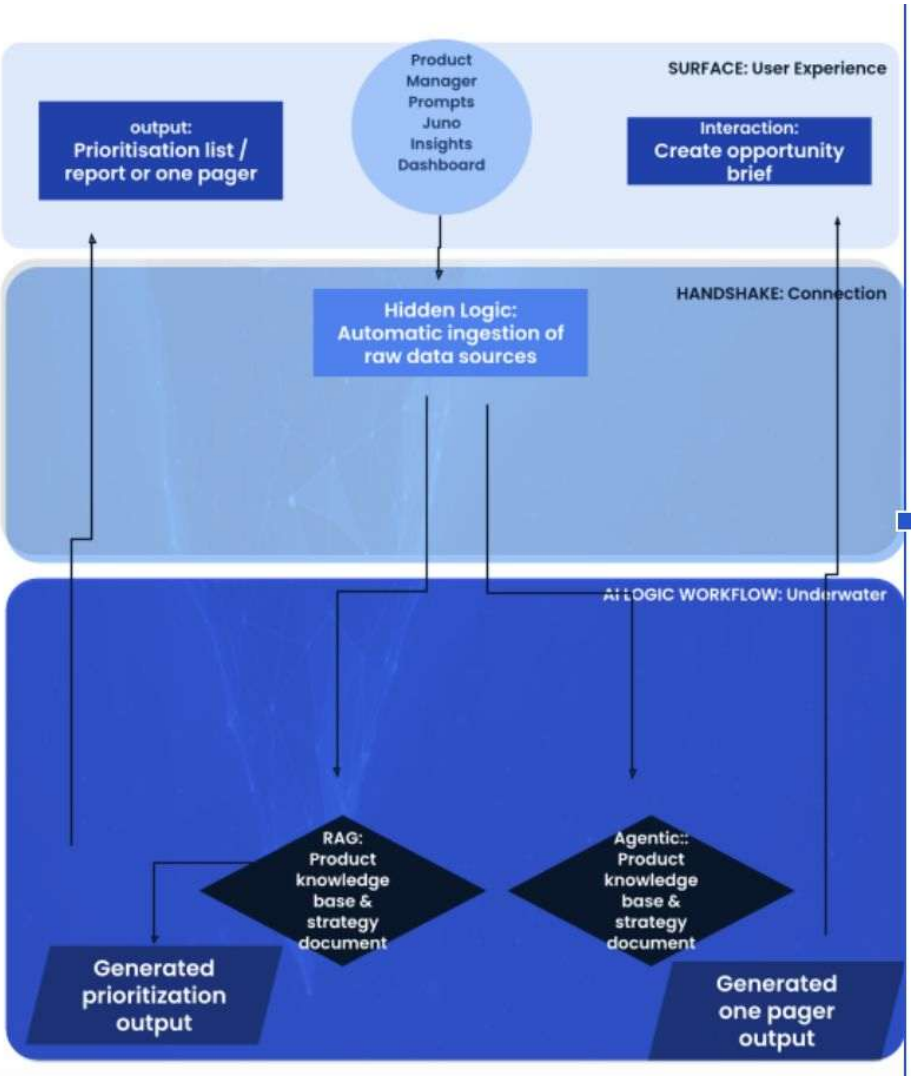
UX Design & Interface

1. SIGNAL — Entry Point (The Trigger):
Product Manager wants to leverage Juno's prioritization and insights framework for their day-to-day tasks. Can be PUSH or PULL.

2. HIDDEN LOGIC — Processing State (The Handshake):
Through User Artifact automation pipeline evaluating Slack channels, folders, JIRA, Meeting Transcripts, CRM Tickets; Juno Insights Dashboard ingests artifacts "automagically" and presents status breadcrumbs. **Data Flow:** RAG review of uploaded product documentation and strategy document. **AI-Generated Output:** Juno Insights Dashboard evaluates top 8 user artifacts to assess updated prioritized list of features.

3. INTERACTION MANEUVER — Presentation Layer:
Juno provides prioritization score based on Strategy Document and ingested artifacts, creating Opportunity Brief with structured insights (what happens in the 3rd and 4th column in Juno).

4. FEEDBACK LOOP — Learning Cycle:
Data Flow: When the PM overrides a prioritisation score, dismisses an insight, or edits the Strategy Document, Juno captures that action as a preference signal and re-evaluates all artifacts against the updated context. **AI-Generated Output:** The Insights Dashboard produces an updated prioritised feature list that reflects both new data and PM feedback — a living prioritisation that improves with every interaction.

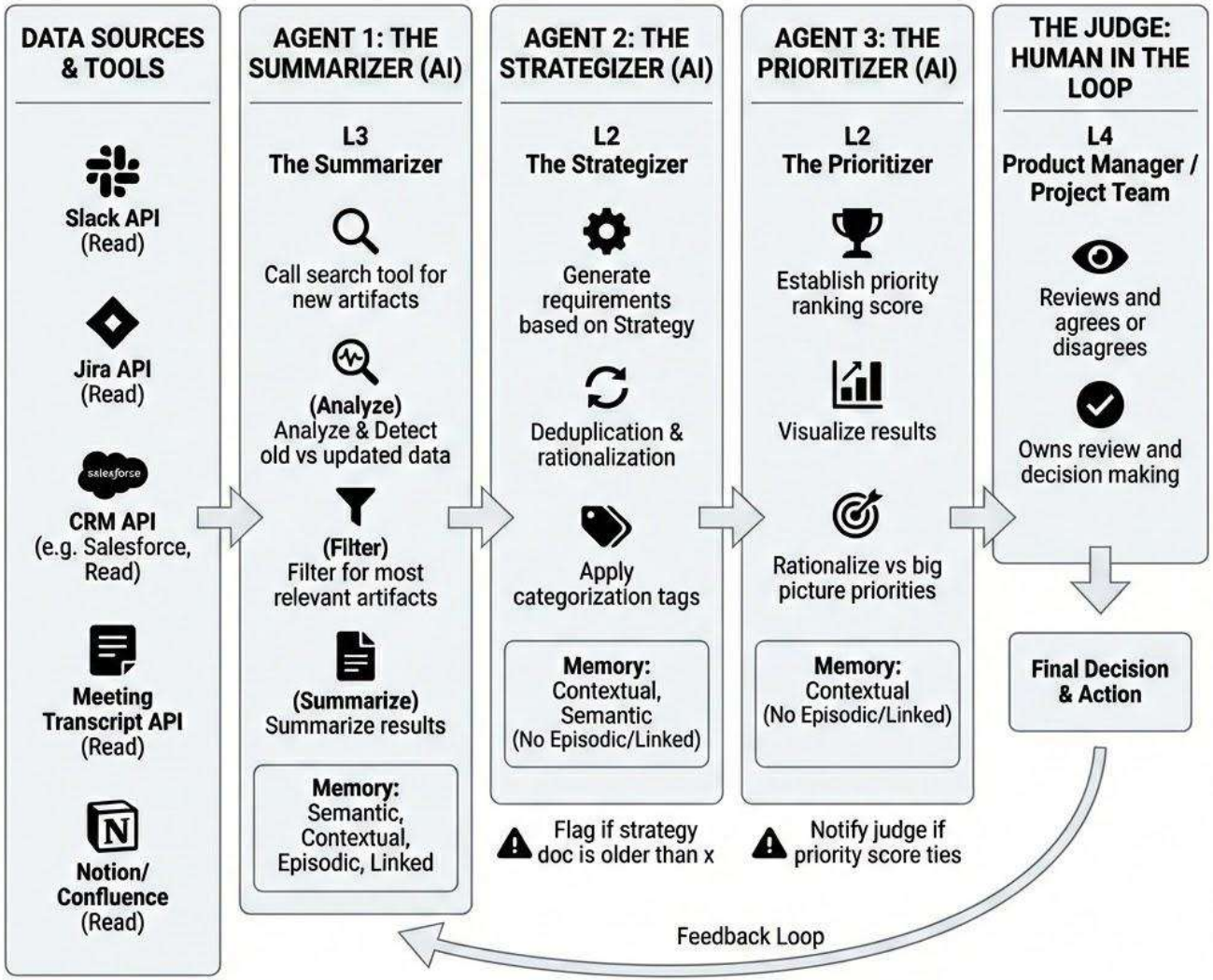


Agent Workflow Spec (AWSpec)

Completed Sections

- **Actors: THE PILOTS**: 1. **L3 The Summarizer** (AI) — compiles raw data from various sources following frequency logic and detects old vs updated data. 2. **L2 The Strategizer** (AI) — analyzes data to filter the most relevant user artifacts and determines which align with the cross-referenced RocketShip strategy. 3. **L2 The Prioritizer** (AI) — recommends where the prioritization should be so the **human in the loop** reviews and agrees or disagrees. **THE JUDGE**: 4. (Human) Product Manager/Project Team — owns the **human in the loop** review and decision making.
- **Pattern Plan:** The Analyzer (**Planner-Executor**): 1. Call search tool to look for new user artifacts. 2. Analyze new user artifacts. 3. Filter artifacts to identify most relevant. 4. Summarize results of reviewed user artifacts. The Strategizer (**Planner-Executor**): 1. Generate requirements based on RocketShip Strategy and identified user artifacts. 2. Deduplication and rationalization of each requirement. 3. Application of categorization/classification tags to improve visualization and rationale for prioritization. The Prioritizer (**Planner-Executor**): 1. Establish a priority ranking level/score for each requirement. 2. Visualize the results so they can be easily understood (e.g. tags, emojis). 3. Rationalize in relation to big picture priorities — OKRs/KPIs.
- **Memory:** The Analyzer: Semantic, Contextual, Episodic, Linked (takes all data in). The Strategizer: Contextual, Semantic (not Episodic — needs big picture, not Linked — uses Analyzer's output). The Prioritizer: Contextual (not Episodic — needs big picture, not Linked — uses Strategizer's output). **Controls**: Stop if no new documents. Stop if data failure or bad data quality. Stop if confidence drops below 70%. Anonymize any PII. Stop if duplicate identified. Stop if prioritization framework and artifacts don't align — notify judge. Do not create new OKRs/KPIs. Notify judge if prioritization score ties. Flag if strategy document is older than x.
- **Tools:** Slack API (read only), Jira API (read only), CRM API e.g. Salesforce (read only), Meeting Transcript API (read only), Notion/Confluence for latest RocketShip strategy documents (read only), Dashboard API (write) — can visualize priority list and insight cards with tags/emojis on the Juno interface.

Agent Workflow Spec (AWSpec)



Evaluation Plan

Definition of “Good”

> **Grounded in evidence.** Every recommendation traces back to a named source - a support ticket, a user transcript, a Slack thread, a CRM signal. If the evidence isn't there, the recommendation isn't made.
> **Connected to strategy.** Outputs are aligned to RocketShip strategy. Juno flags outputs where those connections cant be made.
> **Drives Actionability.** The output tells a PM what to do this sprint while also feeding a priortized backlog. Juno uses visual cues to make the output clear and intuitive.

Eval Methodology

➔ **AI Eval Stack (3 layers): User Feedback:** Active (thumbs up/down after each Juno output in Slack) + Passive (track whether PM edits Juno’s draft before sending - **Human Override Rate).**

Human Evaluations: 3 rubric questions

- (a) Does the decision brief accurately capture the key stakeholders, blockers, and recommended action from the source thread?
- (b) Does the V0.1 PRD include all edge cases and technical constraints surfaced in the source tickets?
- (c) Are the flagged risks ranked by actual business impact, not just frequency?

Automated Assessment: Binary hard gate - Does every speculative claim include a confidence label ([INFERENCE] or [ASSUMPTION])? Yes/No. Automatable via script.

Risk Mitigation

➔ ***Confidence labeling as a hard gate:*** *Juno must tag every inference or assumption inline with [INFERENCE] or [ASSUMPTION]. If the automated check finds any speculative claim without a label, the output is blocked before it reaches the PM. This prevents hallucinated blockers from being treated as confirmed facts. Combined with the Data Grade system from the AWSpec: if confidence drops below 90%, the agent stops and asks the PM for clarification rather than proceeding with uncertain data.*

Group Insights: Building Process

Friction Points

- Deciding what each AI Pilot can do (ex. too broad and the agent hallucinates)
- Deciding how much data Juno should take in (ex. Too much and it's costly; too little and we don't capture everything)
- Where does the PM's job end and engineering's begin, especially for non-technical PMs
- Understanding some of the more complex technical nuance and its impacts on design

Key Learnings

- Ability to use multiple AI agents to focus on specific and separate tasks, which is actually recommended for less drift in thinking/resposnes
- AI requirements and guardrails already differ based on the feature, and through team conversations it's clear that they differ massively between industries, highlighting importance for cross-functional team support and prevention of copy-paste guardrail standards

Aha! Moment

- Using AI to inform AND to develop AI product (Ex. being able to use AI to generate prompts for Juno)
- Push towards the importance for technical experience ("low code" knowledge) for PMs
- Difference between frameworks presented in the course vs real life experimentation - pushes us to analyze what works for us, what doesn't and what can be changed to apply

Extra slide – link to all group lab docs

- Module 1 – different format (we didn't create a shared word doc) https://docs.google.com/document/d/1BpDYOeeaV7KEKsxpIRkIFFuEXNuC0WsJ/edit?usp=drive_link&ouid=110788640034300702694&rtpof=true&sd=true
- Module 2 - https://docs.google.com/document/d/1Rm1C7I_y4aA8fGTzQJjxSbwFcZ3VTlqWbQTCMcyRM2Q/edit?usp=drive_link
- Module 3 - https://docs.google.com/document/d/1wrVIOo6fj79QF0lQqMVFA_AA2kSZ67eYZw-NYcH0MPk/edit?usp=drive_link
- Module 4 - https://docs.google.com/document/d/1DdTJBt-_Dpc9hFeld8_DPki_hi0yPGfWJXpOUCby6ml/edit?usp=drive_link
- Module 5 - https://docs.google.com/document/d/1oD7PXzg4pOkWPAfNF7cW6WGfMi4Qqmb6KjfgpdTcgto/edit?usp=drive_link
- Module 6 - https://docs.google.com/document/d/1IBvvKv11Sy2zEGRx8N88vY93CFtwn3Dt/edit